⚙️ TiDPI 05 — Automatyzacja kompilacji, uruchomienia i rejestracji wyniku

📔 Tematy do dziennika

```
[TiDPI] Kompilacja programu C++ z wiersza poleceń – środowisko MSVC, etapy budowania i podstawowe przełączniki kompilatora
[TiDPI] Automatyzacja przebiegu build → run → evidence w Pythonie z wykorzystaniem modułu subprocess
[TiDPI] Rejestrowanie standardowego wyjścia, standardowego wyjścia błędów i kodów zakończenia oraz dokumentowanie wiarygodnego przebiegu testu
```

🎯 Cel lekcji

Po tej lekcji będziecie potrafili zbudować program C++ bez klikania przycisku w Visual Studio, uruchomić go ze skryptu Pythona, rozpoznać powodzenie lub awarię na podstawie kodu zakończenia oraz automatycznie zachować komplet podstawowych dowodów przebiegu.

🔁 Od eksperymentu ręcznego do powtarzalnej procedury

W poprzednich lekcjach uruchamialiście program, obserwowaliście wynik i przenosiliście dowody do raportu. Przy jednym przypadku da się to zrobić ręcznie. Przy dwóch konfiguracjach, kilku zestawach danych i wielu powtórzeniach człowiek zaczyna jednak wykonywać pracę godną bardzo cierpliwego robota — a potem omyłkowo wkleja wynik z wtorku do tabeli ze środy.

Automatyzacja nie oznacza jeszcze „sztucznej inteligencji, która sama testuje wszystko”. W tej lekcji oznacza coś znacznie bardziej konkretnego: zapisujemy procedurę, która za każdym razem wykonuje te same kroki w tej samej kolejności.

```
kod źródłowy
      ↓
kompilacja i konsolidacja
      ↓
czy powstał nowy program?
   ↙ nie       tak ↘
zapis błędu      uruchomienie
                       ↓
       zapis stdout, stderr i kodu zakończenia
                       ↓
                katalog dowodów
```

W skrócie będziemy budować przebieg:

```
build → run → evidence
```

Słowo **build** oznacza zbudowanie programu, **run** — jego uruchomienie, a **evidence** — dowody pozwalające sprawdzić, co naprawdę się wydarzyło.

> Ręczny test brzmi czasem: „Uruchomiłem i działało”. Automat jest mniej rozmowny, ale ma lepszą pamięć: podaje komendę, trzy pliki z wyjściem, dwa kody zakończenia i dokładnie wie, gdzie je odłożył.

🧪 Co naprawdę robi przycisk „Build”?

Plik `main.cpp` nie jest jeszcze programem wykonywalnym. W uproszczeniu powstaje on w dwóch etapach:

- **kompilacja** tłumaczy kod C++ na kod obiektowy, na przykład `main.obj`;
- **konsolidacja** (*linking*) łączy pliki obiektowe i potrzebne biblioteki w plik wykonywalny, na przykład `experiment.exe`.

W Visual Studio oba etapy uruchamia za Was system budowania. Kompilator Microsoft C/C++ nosi nazwę `cl.exe`. Gdy przekażemy mu plik `.cpp` i nie użyjemy opcji zatrzymującej proces po samej kompilacji, może uruchomić również konsolidator i utworzyć `.exe`.

Nie rezygnujemy z Visual Studio. Zaglądamy tylko pod maskę i uczymy się wywoływać ten sam zestaw narzędzi w sposób, który da się później powtarzać automatycznie.

> Przycisk „Build” nie zawiera małego programisty. Zawiera za to dobrze ustalony ciąg poleceń — a ciągi poleceń są naturalnym środowiskiem życia automatów.

💻 Dlaczego zwykły terminal może nie znać polecenia `cl`?

Visual Studio instaluje kompilator, biblioteki i narzędzia, ale zwykły PowerShell nie musi znać ich położenia. **Developer PowerShell for VS 2022** to terminal uruchomiony z przygotowanymi zmiennymi środowiskowymi. Dzięki nim `cl.exe` wie także, gdzie znaleźć nagłówki i biblioteki.

Otwórzcie z menu Start program **Developer PowerShell for VS 2022**. Następnie sprawdźcie środowisko:

```
cl
python --version
```

Pierwsze polecenie powinno wyświetlić informację o kompilatorze, nawet jeżeli zgłosi brak plików wejściowych. Drugie powinno pokazać wersję Pythona.

Jeżeli pojawi się komunikat, że `cl` nie jest rozpoznawany, nie naprawiajcie kodu C++, ponieważ on nie zdążył jeszcze zrobić niczego złego. Najpierw sprawdźcie, czy terminal jest właściwy oraz czy w Visual Studio Installer zainstalowano zestaw narzędzi C++.

⌨️ Pierwsza kompilacja z wiersza poleceń

Przejdźcie w Developer PowerShellu do katalogu zawierającego `main.cpp`, a następnie wykonajcie:

```
cl /nologo /std:c++17 /EHsc /W4 /permissive- /utf-8 main.cpp /Fe:experiment.exe
```

Jeśli kompilacja i konsolidacja zakończą się powodzeniem, uruchomcie:

```
.\experiment.exe
```

W PowerShellu przed nazwą programu z bieżącego katalogu zapisujemy `.\`. Samo `experiment.exe` może nie zostać odnalezione, nawet gdy plik leży tuż obok.

🎛️ Co oznaczają przełączniki?

Przełącznik	Znaczenie
`/nologo`	ukrywa nagłówek reklamowy kompilatora, dzięki czemu log jest krótszy
`/std:c++17`	wybiera standard języka C++17
`/EHsc`	włącza standardową obsługę wyjątków C++
`/W4`	ustawia wysoki poziom ostrzeżeń
`/permissive-`	wymaga lepszej zgodności ze standardem C++
`/utf-8`	interpretuje pliki źródłowe jako UTF-8
`/Fe:experiment.exe`	określa nazwę wynikowego pliku wykonywalnego

Ostrzeżenie nie jest tym samym co błąd kompilacji. Program może powstać mimo ostrzeżeń, ale automat powinien je zachować. Tester nie zasłania kontrolki taśmą izolacyjną tylko dlatego, że samochód jeszcze jedzie.

🔀 Debug i Release z wiersza poleceń

Na potrzeby tej lekcji użyjemy dwóch uproszczonych zestawów ustawień:

```
cl /nologo /std:c++17 /EHsc /W4 /permissive- /utf-8 main.cpp /Fe:experiment.exe /Od /MDd
cl /nologo /std:c++17 /EHsc /W4 /permissive- /utf-8 main.cpp /Fe:experiment.exe /O2 /MD
```

Istnieje również wariant Release nastawiony na zmniejszenie rozmiaru kodu wynikowego:

```
cl /nologo /std:c++17 /EHsc /W4 /permissive- /utf-8 main.cpp /Fe:experiment.exe /O1 /MD
```

`/Od` wyłącza optymalizację, `/O2` włącza zestaw optymalizacji nastawiony przede wszystkim na szybkość, a `/O1` — zestaw nastawiony na możliwie mały rozmiar wygenerowanego kodu. Bardziej szczegółowe przełączniki `/Ot` i `/Os` mówią kompilatorowi, czy przy wyborze optymalizacji ma preferować odpowiednio szybkość, czy mały kod; są już elementami ustawień wybieranych przez `/O2` i `/O1`.

Określenie „optymalizacja pamięci" byłoby tutaj trochę zbyt szerokie. `/O1` może zmniejszyć plik EXE lub DLL oraz ilość pamięci zajmowanej przez załadowany kod, ale nie zmniejszy automatycznie Waszych tablic, obiektów na stercie ani zapotrzebowania algorytmu na pamięć. Jeżeli program przechowuje milion liczb całkowitych, kompilator nie może po prostu zgubić połowy z nich w imię oszczędności.

`/MDd` wybiera debugową, wielowątkową i dynamicznie dołączaną wersję biblioteki uruchomieniowej MSVC. Ten przełącznik sam powoduje zdefiniowanie makr `_DEBUG`, `_MT` oraz `_DLL`, dlatego dodatkowe `/D_DEBUG` byłoby tutaj zbędne.

`/MD` wybiera zwykłą, wielowątkową i dynamicznie dołączaną wersję biblioteki uruchomieniowej. Definiuje `_MT` oraz `_DLL`, ale nie definiuje makra będącego prostym przeciwieństwem `_DEBUG`. W tym eksperymencie nie potrzebujemy takiego makra: konfigurację wybiera skrypt, a w podsumowaniu zapisuje ją jawnie jako `debug` albo `release`.

Nie traktujcie tych kilku przełączników jako pełnej definicji konfiguracji Visual Studio. Projekt może zawierać wiele dodatkowych ustawień. W naszym małym eksperymencie świadomie zapisujemy minimalny, jawny przepis.

📡 Trzy głosy jednego procesu

Uruchomiony program komunikuje się z otoczeniem co najmniej trzema ważnymi kanałami:

- **standardowe wyjście** (*standard output*, `stdout`) — zwykły wynik działania;
- **standardowe wyjście błędów** (*standard error*, `stderr`) — diagnostyka i komunikaty o problemach;
- **kod zakończenia** (*exit code*, *return code*) — liczba przekazana systemowi po zakończeniu procesu.

Umownie kod `0` oznacza powodzenie, a kod różny od zera — niepowodzenie albo szczególny stan wymagający interpretacji. Znaczenie kodów powinien określić autor programu. W naszym przykładzie przyjmujemy:

Kod	Znaczenie
`0`	wynik testu jest poprawny
`1`	wynik programu różni się od wyniku wzorcowego
`2`	przekazano nieznany argument

Po ręcznym uruchomieniu programu w PowerShellu ostatni kod zakończenia możecie odczytać tak:

```
$LASTEXITCODE
```

Sam tekst na ekranie nie wystarczy. Program może wypisać słowo `ERROR` i zakończyć się kodem `0`, albo milczeć i zwrócić kod `1`. Automat powinien znać umowę i sprawdzać kanał przeznaczony do sygnalizowania wyniku.

Przekierowanie — pierwsza metoda zapisu wyniku

PowerShell potrafi skierować zwykłe wyjście programu do pliku:

```
.\experiment.exe > run_stdout.txt
```

Można też oddzielnie zapisać strumień błędów:

```
.\experiment.exe 1> run_stdout.txt 2> run_stderr.txt
```

Numer `1` oznacza `stdout`, a numer `2` — `stderr`. To użyteczne przy pojedynczym uruchomieniu, ale nadal trzeba ręcznie skompilować program, sprawdzić kod zakończenia, nadać plikom nazwy i nie pomylić przebiegów. Dlatego kolejnym krokiem będzie Python.

`subprocess` — Python uruchamia inny program

Moduł `subprocess` pozwala programowi w Pythonie uruchomić inny proces i odebrać jego wynik. Minimalny przykład wygląda tak:

```
result = subprocess.run(
    ["experiment.exe"],
    capture_output=True,
    check=False,
)
```

Lista `['experiment.exe']` opisuje program i jego argumenty. `capture_output=True` przechwytuje `stdout` i `stderr`, a `check=False` oznacza, że kod różny od zera nie zostanie natychmiast zamieniony w wyjątek. Możemy go świadomie zapisać i zinterpretować jako wynik testu.

W naszym automacie użyjemy `shell=False`. Python uruchomi wskazany program bez pośrednictwa powłoki. Lista argumentów nie wymaga ręcznego sklejania i cytowania całej komendy, a dane nie zostaną przypadkiem potraktowane jak polecenia powłoki.

Nie uruchamiajcie w ten sposób kodu ani poleceń z nieznanego źródła. Automat wykonuje działania szybciej od człowieka, włącznie z działaniami nierozsądnymi.

Najgroźniejszy sukces: uruchomienie starego EXE

Wyobraźcie sobie taki przebieg:

- wczoraj program skompilował się poprawnie i powstał `experiment.exe`;
- dzisiaj wprowadziliście błąd składni;
- kompilacja się nie udała, ale stary `experiment.exe` nadal leży w katalogu;
- wadliwy skrypt bez sprawdzenia uruchamia wczorajszy program;
- na ekranie pojawia się poprawny wynik i skrypt ogłasza sukces.

To **fałszywie dodatni wynik automatyzacji**: procedura informuje o powodzeniu, chociaż bieżącego kodu w ogóle nie zbudowano i nie przetestowano.

Dlatego nasz skrypt przed kompilacją usuwa poprzedni plik wykonywalny, a program uruchamia tylko wtedy, gdy jednocześnie:

- kompilator zwrócił kod `0`;
- nowy plik `.exe` rzeczywiście istnieje.

> Stary EXE jest jak kanapka znaleziona w szkolnym plecaku po wakacjach: fakt, że istnieje, nie dowodzi, że powstała dzisiaj ani że należy jej ufać.

💻 Program C++ używany w eksperymencie

Program sortuje niewielki, stały zestaw danych i porównuje wynik z `std::sort`. Bez argumentów kończy się sukcesem. Argument `--simulate-failure` celowo psuje wynik po sortowaniu, abyśmy mogli bezpiecznie sprawdzić ścieżkę awarii automatu.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

void bubble_sort(std::vector<int>& values)
{
    for (std::size_t pass = 0; pass + 1 < values.size(); ++pass)
    {
        bool swapped = false;

        for (std::size_t i = 0; i + 1 < values.size() - pass; ++i)
        {
            if (values[i] > values[i + 1])
            {
                std::swap(values[i], values[i + 1]);
                swapped = true;
            }
        }

        if (!swapped)
        {
// Brak zamiany w całym przebiegu oznacza, że dane są już uporządkowane,
            // więc następne przebiegi nie zmienią wyniku.
            break;
        }
    }
}

int main(int argc, char* argv[])
{
    bool simulate_failure = false;

    if (argc == 2 && std::string(argv[1]) == "--simulate-failure")
    {
        simulate_failure = true;
    }
    else if (argc != 1)
    {
        std::cerr << "ERROR: unknown argument\n";
        return 2;
    }

    const std::vector<int> source{ 8, 3, 7, 4, 9, 2, 6, 1 };
    std::vector<int> expected = source;
    std::vector<int> result = source;

    std::sort(expected.begin(), expected.end());
    bubble_sort(result);

    if (simulate_failure)
    {
// Celowo psujemy wynik, aby sprawdzić nie algorytm, lecz reakcję automatyzacji:
        // czy wykryje niepowodzenie i zachowa właściwe dowody.
        result.front() = -1;
    }
```

```
    const bool correct = result == expected;

    std::cout
        << "Case: " << (simulate_failure ? "simulated failure" : "normal") << '\n'
        << "Input count: " << source.size() << '\n'
        << "Result correct: " << std::boolalpha << correct << '\n';

    if (!correct)
    {
        std::cerr << "ERROR: result differs from the reference\n";
    }

// Kod wyjścia 0 oznacza sukces, a inna wartość błąd. Dzięki temu skrypt może
    // podjąć decyzję bez zgadywania znaczenia tekstu przeznaczonego dla człowieka.
    return correct ? 0 : 1;
}
```

Symulacja błędu nie udaje prawdziwego odkrycia. Jest **testem negatywnym naszej automatyzacji**: świadomie wywołujemy awarię i sprawdzamy, czy procedura ją zauważy, zachowa dowody oraz sama zakończy się kodem różnym od zera.

🤖 Kompletny automat w Pythonie

Zapiszcie poniższy skrypt jako `run_experiment.py` obok `main.cpp`.

```
"""Skompiluj eksperyment C++, uruchom go i zachowaj dowody z obu etapów."""

import argparse
import datetime
import locale
import shutil
import subprocess
import time
from pathlib import Path


PROJECT_DIR = Path(__file__).resolve().parent
SOURCE = Path("main.cpp")
BUILD_DIR = Path("_build")
PROGRAM = BUILD_DIR / "experiment.exe"
EVIDENCE_ROOT = PROJECT_DIR / "logs"


def decode_output(data):
    """Odczytaj wynik narzędzi, które mogą używać różnych kodowań znaków.

    Args:
        data: Bajty przechwycone ze standardowego wyjścia albo wyjścia błędów.

    Returns:
        Odczytany tekst. Niepoprawne bajty są zastępowane dopiero po
        sprawdzeniu kodowań typowych dla używanych narzędzi Windows.
    """
    if not data:
        return ""

    encodings = ("utf-8", locale.getpreferredencoding(False), "cp852")
    for encoding in dict.fromkeys(encodings):
        try:
            return data.decode(encoding)
        except UnicodeDecodeError:
```

```
            pass

    return data.decode("utf-8", errors="replace")


def save_text(path, content):
    """Ujednolić końce wierszy i zastąpić plik tekstowy UTF-8.

    Args:
        path: Ścieżka pliku docelowego.
        content: Tekst, który ma zostać ujednolicony i zapisany.
    """
    normalized = content.replace("\r\n", "\n").replace("\r", "\n")
    path.write_text(normalized, encoding="utf-8")


def save_summary(path, values):
    """Zapisać dowody jako uporządkowane pary klucz-wartość.

    Args:
        path: Ścieżka pliku docelowego.
        values: Mapa, której kolejność dodawania ustala kolejność wierszy.
    """
    lines = [f"{key}={value}" for key, value in values.items()]
    save_text(path, "\n".join(lines) + "\n")


def run_process(command, timeout_seconds):
    """Uruchomić polecenie, przechwycić dowody i pilnować limitu czasu.

    Args:
        command: Lista argumentów przekazywana bezpośrednio procesowi potomnemu.
        timeout_seconds: Maksymalny czas działania w sekundach.

    Returns:
        Kod wyjścia, bajty stdout i stderr, czas w milisekundach oraz
        informacja o przekroczeniu czasu. Kod 124 oznacza timeout.
    """
    start = time.perf_counter()

    try:
        completed = subprocess.run(
            command,
            cwd=PROJECT_DIR,
            capture_output=True,
            shell=False,
            timeout=timeout_seconds,
            check=False,
        )
        elapsed_ms = (time.perf_counter() - start) * 1000
        return completed.returncode, completed.stdout, completed.stderr, elapsed_ms, False
    except subprocess.TimeoutExpired as error:
        elapsed_ms = (time.perf_counter() - start) * 1000
        return 124, error.stdout or b"", error.stderr or b"", elapsed_ms, True


def main():
    """Skompilować i uruchomić wybrany przypadek oraz zwrócić jego status."""
    parser = argparse.ArgumentParser(
        description="Compile, run and register evidence from a C++ experiment."
    )
    parser.add_argument(
        "--configuration",
```

```
        choices=("debug", "release"),
        default="release",
    )
    parser.add_argument(
        "--case",
        choices=("pass", "fail"),
        default="pass",
    )
    args = parser.parse_args()

    # Dodajemy mikrosekundy, aby dwa szybko wykonane uruchomienia nie zapisały
    # dowodów w tym samym katalogu i nie nadpisały swoich wyników.
    run_id = datetime.datetime.now().strftime("%Y%m%d_%H%M%S_%f")
    evidence_dir = EVIDENCE_ROOT / run_id
    evidence_dir.mkdir(parents=True)
    (PROJECT_DIR / BUILD_DIR).mkdir(exist_ok=True)

    summary = {
        "run_id": run_id,
        "configuration": args.configuration,
        "case": args.case,
        "compile_exit_code": "not_started",
        "execution_attempted": "false",
        "run_exit_code": "not_started",
        "final_status": "started",
    }

    if shutil.which("cl") is None:
        # Brak cl.exe oznacza źle przygotowane środowisko, a nie błąd badanego programu.
        # Rozróżnienie tych sytuacji zapobiega niesprawiedliwemu wynikowi testu.
        summary["final_status"] = "environment_error"
        save_summary(evidence_dir / "summary.txt", summary)
        print("ERROR: cl.exe not found.")
        print("Run this script from Developer PowerShell for VS 2022.")
        return 2

    program_path = PROJECT_DIR / PROGRAM
    # Usuwamy stary plik EXE, aby po nieudanej kompilacji nie uruchomić przypadkiem
    # programu pozostałego z poprzedniej, udanej próby.
    program_path.unlink(missing_ok=True)

    # Przekazujemy argumenty jako listę i nie uruchamiamy powłoki. Python nie musi
    # wtedy interpretować cudzysłowów ani znaków specjalnych polecenia.
    compile_command = [
        "cl",
        "/nologo",
        "/std:c++17",
        "/EHsc",
        "/W4",
        "/permissive-",
        "/utf-8",
        str(SOURCE),
        f"/Fe:{PROGRAM}",
        f"/Fo:{BUILD_DIR / 'main.obj'}",
    ]

    if args.configuration == "release":
        compile_command.extend(("/O2", "/MD"))
    else:
        compile_command.extend(("/Od", "/MDd"))

    summary["compile_command"] = subprocess.list2cmdline(compile_command)
    compile_code, compile_stdout, compile_stderr, compile_ms, compile_timeout = (
```

```
        run_process(compile_command, timeout_seconds=60)
    )
    save_text(evidence_dir / "compile_stdout.txt", decode_output(compile_stdout))
    save_text(evidence_dir / "compile_stderr.txt", decode_output(compile_stderr))
    summary["compile_exit_code"] = compile_code
    summary["compile_time_ms"] = f"{compile_ms:.3f}"
    summary["compile_timeout"] = str(compile_timeout).lower()

    # Sama wartość 0 nie wystarcza: sprawdzamy też, czy powstał plik EXE. Dzięki temu
    # uszkodzone narzędzie lub nietypowa konfiguracja nie zgłosi fałszywego sukcesu.
    if compile_code != 0 or not program_path.exists():
        summary["final_status"] = "compile_failed"
        save_summary(evidence_dir / "summary.txt", summary)
        print(f"COMPILE FAILED. Evidence: logs/{run_id}")
        return compile_code if compile_code != 0 else 1

    run_command = [str(PROGRAM)]
    if args.case == "fail":
        run_command.append("--simulate-failure")

    summary["execution_attempted"] = "true"
    summary["run_command"] = subprocess.list2cmdline(run_command)
    run_code, run_stdout, run_stderr, run_ms, run_timeout = run_process(
        run_command,
        timeout_seconds=10,
    )
    save_text(evidence_dir / "run_stdout.txt", decode_output(run_stdout))
    save_text(evidence_dir / "run_stderr.txt", decode_output(run_stderr))
    summary["run_exit_code"] = run_code
    summary["run_time_ms"] = f"{run_ms:.3f}"
    summary["run_timeout"] = str(run_timeout).lower()
    summary["final_status"] = "passed" if run_code == 0 else "run_failed"
    save_summary(evidence_dir / "summary.txt", summary)

    print(f"{summary['final_status'].upper()}. Evidence: logs/{run_id}")
    return run_code


if __name__ == "__main__":
    raise SystemExit(main())
```

🔍 Jak działa automat?

Skrypt tworzy identyfikator przebiegu z daty i czasu, a następnie osobny katalog `logs/<run_id>`. Dzięki temu nowe uruchomienie nie nadpisuje dowodów poprzedniego.

`shutil.which('cl')` sprawdza, czy kompilator jest dostępny w środowisku. Jeżeli go brakuje, skrypt nie udaje błędu badanego programu, lecz zapisuje `environment_error` — problem środowiska.

`program_path.unlink(missing_ok=True)` usuwa poprzedni plik EXE. Dopiero później wykonywana jest kompilacja. Polecenie jest listą argumentów, a jego czytelna postać trafia do `summary.txt`.

Funkcja `run_process` mierzy czas procesu, przechwytuje oba strumienie i zwraca kod zakończenia. Limit czasu chroni automat przed programem, który zawiesił się albo działa znacznie dłużej, niż przewidywał eksperyment. Kod `124` jest tutaj naszą umową oznaczającą przekroczenie limitu.

Wyjście przechwytujemy początkowo jako bajty. Funkcja `decode_output` próbuje kilku typowych kodowań Windows, a `save_text` ujednolica zakończenia wierszy i zapisuje pliki dowodowe w UTF-8. Jest to praktyczny kompromis: narzędzia uruchomione w różnych terminalach nie zawsze mówią tym samym kodowaniem.

Na końcu `raise SystemExit(main())` przekazuje kod zwrócony przez `main` do systemu. Dzięki temu niepowodzenie badanego programu staje się również niepowodzeniem skryptu. To ważne, gdy kiedyś skrypt uruchomi następny automat, na przykład system CI.

▶️ Uruchamianie scenariuszy

Uruchomienie podstawowe kompiluje program w konfiguracji Release i wykonuje przypadek poprawny:

```
python .\run_experiment.py
```

Jawny zapis tego samego wariantu:

```
python .\run_experiment.py --configuration release --case pass
```

Konfiguracja Debug:

```
python .\run_experiment.py --configuration debug --case pass
```

Kontrolowana symulacja niepowodzenia:

```
python .\run_experiment.py --configuration release --case fail
```

Po ostatnim poleceniu `run_exit_code` i kod samego skryptu powinny wynosić `1`. To oczekiwany wynik testu negatywnego — automat właśnie udowodnił, że potrafi zauważyć awarię.

📁 Co znajdzie się w katalogu dowodów?

Poprawnie rozpoczęty przebieg tworzy strukturę podobną do tej:

```
logs/
└── 20260815_123456_123456/
    ├── compile_stdout.txt
    ├── compile_stderr.txt
    ├── run_stdout.txt
    ├── run_stderr.txt
    └── summary.txt
```

Data w przykładzie jest fikcyjna. Wasz identyfikator będzie odpowiadał chwili rzeczywistego uruchomienia.

Plik `summary.txt` może wyglądać tak:

```
run_id=20260815_123456_123456
configuration=release
case=pass
compile_exit_code=0
execution_attempted=true
run_exit_code=0
final_status=passed
compile_command=cl /nologo ...
compile_time_ms=742.315
compile_timeout=false
run_command=_build\experiment.exe
run_time_ms=18.402
run_timeout=false
```

To są tylko przykład formatu. Czasy zależą od komputera i aktualnego obciążenia. Pojedynczego `compile_time_ms` ani `run_time_ms` nie traktujemy jako benchmarku — rejestrujemy go jako metadane przebiegu.

Przy błędzie kompilacji pliki `run_stdout.txt` i `run_stderr.txt` nie powstaną, ponieważ uruchomienie nie zostało podjęte. `execution_attempted=false` jest wówczas cennym dowodem prawidłowego zachowania automatu, a nie brakiem, który trzeba ukryć.

📜 Log nie jest jeszcze raportem

Log jest surowym zapisem zdarzeń lub wyjścia programu. **Raport** wybiera istotne dowody, opisuje procedurę, interpretuje wynik i podaje ograniczenia. Wklejenie całego logu bez komentarza nie zastępuje analizy, tak jak nagranie z kamery samochodowej nie zastępuje wyjaśnienia, dlaczego doszło do kolizji.

Zachowujemy surowe pliki, ponieważ pozwalają:

- sprawdzić, czy raport nie pomija niewygodnego komunikatu;
- odróżnić błąd kompilacji od błędu wykonania;
- odtworzyć użyte polecenie;
- porównać przebiegi bez polegania na pamięci;
- przekazać jednoznaczne dowody innej osobie lub agentowi analizującemu raport.

W tej lekcji rejestrujemy wynik, lecz nie budujemy jeszcze rozbudowanego parsera. W kolejnej lekcji uporządkujemy dane wyjściowe i nauczymy się sprawdzać ich strukturę automatycznie.

♾️ Od skryptu do CI

Continuous Integration (`CI`), czyli ciągła integracja, to praktyka regularnego łączenia zmian oraz automatycznego sprawdzania projektu. Usługa CI może po każdej zmianie w repozytorium wykonać podobny przebieg:

```
pobranie kodu → przygotowanie środowiska → build → testy → artefakty i raport
```

Nasz skrypt nie jest jeszcze całym CI. Jest jednak jego lokalnym zalążkiem: ma jasno określone kroki, nie wymaga klikania, zwraca kod zakończenia i zachowuje artefakty. Te same cechy pozwalają później uruchomić go na szkolnym komputerze, serwerze budującym albo runnerze GitHub Actions.

> CI jest jak niezwykle sumienny członek zespołu: uruchamia tę samą procedurę po każdej zmianie, nie mówi „przecież wczoraj działało” i nigdy nie zapomina dołączyć logów. Za to sam nie wyjaśni, czy testowaliśmy właściwą rzecz — od tego nadal jesteśmy my.

🧪 Ćwiczenia i eksperyment

Wykonujcie poziomy kolejno. Każdy przebieg zachowujcie w osobnym katalogu, a dowody oznaczajcie identyfikatorami `D01`, `D02` i tak dalej. Nie zmieniajcie ręcznie zawartości logów; komentarz i interpretacja należą do raportu.

🟢 Poziom A — buduję i rejestruję poprawny przebieg

- Przygotujcie `main.cpp` oraz `run_experiment.py` zgodnie z listingami.
- W Developer PowerShellu potwierdźcie dostępność `cl` i Pythona.
- Skompilujcie i uruchomcie program ręcznie. Zapiszcie użyte polecenia i kod zakończenia.
- Uruchomcie automat dla `release/pass`.
- Pokażcie katalog dowodów i wyjaśnijcie rolę każdego pliku.
- Powiążcie `final_status=passed` z kodem kompilacji, istnieniem nowego EXE i kodem programu.
- Uruchomcie ten sam wariant drugi raz i potwierdźcie, że powstał nowy katalog, a poprzednie dowody nie zostały nadpisane.

📦 *Do oddania — poziom A*

Przekażcie poniższy zestaw plików. W miejscu Kryptonim wpiszcie Wasz przydzielony kryptonim TiDPI.

```
Kryptonim\
└── TiDPI_05\
    ├── raport.md
    ├── main.cpp
    ├── run_experiment.py
    ├── logs\
    │   └── <run_id>\  (osobno dla każdego z dwóch przebiegów pass)
    │       ├── compile_stdout.txt
    │       ├── compile_stderr.txt
    │       ├── run_stdout.txt
    │       ├── run_stderr.txt
    │       └── summary.txt
    └── images\  (jeśli używacie obrazów)
```

Przekażcie oba odrębne katalogi automatycznych przebiegów release/pass, łącznie z pustymi plikami strumieni, jeśli takie powstały. W raporcie zapiszcie ręczne polecenia i kod zakończenia, potwierdzenie dostępności narzędzi oraz objaśnienie dowodów. Zachowajcie układ main.cpp obok run_experiment.py, którego oczekuje listing. Oddajecie jedno sprawozdanie z oznaczeniem faktycznie wykonanego zakresu. Nazwy dodatkowych plików pokazane w drzewie są wzorcem organizacyjnym; jeżeli Wasz program używa innych nazw, zachowajcie działające odwołania i wskażcie te pliki w raporcie. Zapis <...> oznacza nazwę do zastąpienia, a nie dosłowną nazwę pliku. Wszystkie obrazy użyte w raporcie umieśćcie w images\ i odwołujcie się do nich względnie, np. ![Dowód D01](images/D01.png). Jeżeli nie używacie obrazów, katalog images\ nie jest wymagany. Pozostałe załączniki również linkujcie względnie.

🛡️ Poziom B — sprawdzam ścieżki niepowodzenia

- Uruchomcie wariant `release/fail` i wykażcie, że komunikat błędu trafił do `run_stderr.txt`, zwykły wynik do `run_stdout.txt`, a kod zakończenia wynosi `1`.
- Wprowadźcie kontrolowany błąd składni w `main.cpp` i uruchomcie skrypt.
- Wykażcie na podstawie `summary.txt`, że kompilacja się nie udała, a program nie został uruchomiony.
- Sprawdźcie, czy po nieudanej kompilacji nie pozostał plik `_build/experiment.exe`.
- Przywróćcie poprawny kod i ponownie uzyskajcie przebieg `passed`.

- Wyjaśnijcie różnicę między `environment_error`, `compile_failed` i `run_failed`.

Ukończenie wyłącznie poziomów A i B pozwala uzyskać najwyżej ocenę 4.

📦 *Do oddania — poziom B*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_05\
    ├── raport.md  (uzupełniony o A–B)
    ├── main.cpp  (po przywróceniu poprawnego kodu)
    └── logs\
        ├── <run_id_fail>\  (komplet pięciu plików jak w A)
        ├── <run_id_compile_failed>\
        │   ├── compile_stdout.txt
        │   ├── compile_stderr.txt
        │   └── summary.txt
        └── <run_id_pass_po_naprawie>\  (komplet pięciu plików jak w A)
```

Nazwy katalogów zachowajcie takie, jakie nadał automat; przyporządkujcie je do przypadków w rejestrze dowodów. Przy błędzie kompilacji nie dopisujcie sztucznych run_stdout.txt i run_stderr.txt, ponieważ program nie był uruchamiany. W raporcie pokażcie kod 1 dla fail, execution_attempted=false, brak starego EXE i ponowne powodzenie po naprawie.

🔧 Poziom C — rozwijam automat

Wykonajcie dwa z trzech rozszerzeń:

- **Wiele powtórzeń.** Dodajcie argument `--repeat` przyjmujący liczbę od `1` do `10`. Każde uruchomienie programu ma otrzymać osobne pliki `run_01_stdout.txt`, `run_01_stderr.txt` itd. W podsumowaniu zachowajcie kod każdej próby i nie przerywajcie zapisu dowodów po pierwszej awarii.
- **Kontrolowane wejście programu.** Rozbudujcie program C++ o argument `--size`, a skrypt o odpowiadający mu parametr. Walidujcie zakres przed uruchomieniem i zachowajcie przekazaną wartość w `summary.txt`. Nie sklejajcie polecenia w jeden napis wykonywany przez powłokę.
- **Manifest plików.** Dopiszcie do podsumowania rozmiar w bajtach i czas ostatniej modyfikacji pliku źródłowego oraz nowego EXE. Wyjaśnijcie, w czym te metadane pomagają i dlaczego same nie dowodzą zgodności treści pliku.

Poprawne ukończenie poziomów A–C otwiera drogę do oceny 5.

📦 *Do oddania — poziom C*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_05\
    ├── raport.md  (uzupełniony o A–C)
    ├── main.cpp  (wersja dla wybranych rozszerzeń)
    ├── run_experiment.py  (rozbudowany automat)
    └── logs\
        └── <run_id_rozszerzenia>\
            ├── compile_stdout.txt
            ├── compile_stderr.txt
            ├── summary.txt
            ├── <pliki stdout prób>
            └── <pliki stderr prób>
```

Dołączcie dowody sukcesu i niepowodzenia dla dwóch wybranych rozszerzeń. Jeśli wybraliście --repeat, oddajcie run_01_stdout.txt, run_01_stderr.txt i kolejne pary oraz kody każdej próby w podsumowaniu. Przy --size zachowajcie przekazaną wartość i dowód walidacji; przy manifeście — rozmiary i czasy modyfikacji źródła oraz EXE w summary.txt. Pliki strumieni dołączajcie dla faktycznie wykonanych etapów.

📦 *Do oddania — poziom D*

Zachowajcie materiały z wcześniejszych wykonanych poziomów i uzupełnijcie ten sam raport. Poniżej pokazano pliki dodawane lub aktualizowane na tym poziomie.

```
Kryptonim\
└── TiDPI_05\
    ├── raport.md  (uzupełniony o A–D)
    ├── main.cpp
    ├── run_experiment.py  (wersja końcowa)
    ├── <plik konfiguracji JSON/YAML>  (jeśli wybrano konfigurację)
    ├── <definicja zadania CI>  (jeśli wybrano CI)
    └── logs\
        └── <run_id_testu_rozszerzenia>\  (komplet powstałych dowodów)
```

Oddajcie wyłącznie pliki właściwe dla wybranego rozszerzenia i komplet dowodów testów automatu, również przypadków awaryjnych. Skróty SHA-256 albo dodatkowe statusy, jeśli są przedmiotem rozszerzenia, zachowajcie w podsumowaniach. Uzasadnienie projektu i sposób odtworzenia testów zapiszcie w raporcie.

🚀 Poziom D — własna wiarygodna linia testowa

Zaprojektujcie jedno rozszerzenie ponadprogramowe, na przykład:

- obliczanie kryptograficznego skrótu SHA-256 źródła i pliku EXE oraz zapis skrótów w podsumowaniu;
- osobne statusy dla przekroczenia czasu, nieznanego kodu zakończenia i braku artefaktu po pozornie poprawnej kompilacji;
- konfigurację automatu w bezpiecznie odczytywanym pliku JSON lub YAML wraz z walidacją wymaganych pól;
- uruchomienie tego samego skryptu w lokalnym zadaniu CI i zachowanie katalogu `logs` jako artefaktu;
- inny pomysł uzgodniony z prowadzącym.

Ocena 6 wymaga nie tylko większej liczby opcji, lecz samodzielnego problemu, uzasadnionego projektu, obsługi niepowodzeń, dowodów testów samego automatu i umiejętności obrony rozwiązania. Automat mający 300 linii i żadnego sprawdzonego błędu jest po prostu dłuższym sposobem na bycie niepewnym.

📝 Sprawozdanie

Oddajecie jedno indywidualne sprawozdanie w Markdownie zgodne z zasadami lekcji 01. Raport może zostać przekazany internetowemu agentowi AI do wstępnej analizy, dlatego musi być całkowicie zanonimizowany.

🔒 Obowiązkowa anonimizacja

- użyjcie wyłącznie przydzielonego kryptonimu TiDPI;
- nie umieszczajcie imienia, nazwiska, adresu e-mail ani nazwy konta w treści, nazwie pliku, metadanych, ścieżkach, zrzutach i innych dowodach;
- przed dołączeniem logów sprawdźcie, czy polecenia i komunikaty nie zawierają bezwzględnej ścieżki z nazwą konta Windows;
- sprawdźcie właściwości dokumentów DOCX lub PDF i kadrowanie zrzutów;
- mapę `kryptonim TiDPI ↔ uczeń` przechowuje wyłącznie nauczyciel.

🧱 Wymagana struktura raportu

- `Kryptonim TiDPI i wykonany poziom;`
- `Cel oraz kryteria powodzenia automatu;`
- `Metryczka środowiska;`
- `Schemat przebiegu build → run → evidence;`
- `Polecenie kompilacji i znaczenie ustawień;`
- `Procedura;`
- `Przebieg poprawny;`
- `Symulowana awaria programu;`
- `Błąd kompilacji i dowód zatrzymania procedury;`
- `Rejestr dowodów D01, D02…;`
- `Zmiany wykonane w poziomach C lub D;`
- `Interpretacja i ograniczenia;`
- `Wnioski;`
- `Wykorzystanie AI i sposób weryfikacji.`

Do raportu nie musicie wklejać całej zawartości każdego pliku. Umieśćcie czytelny rejestr dowodów, istotne fragmenty oraz komplet surowych artefaktów w miejscu wskazanym przez prowadzącego. Każde twierdzenie o powodzeniu lub awarii powiążcie z konkretnym kodem zakończenia, statusem albo plikiem.

Użycie AI jest dozwolone. Wskażcie zgodnie z prawdą narzędzie, zakres pomocy i sposób sprawdzenia kodu oraz wniosków. Musicie potrafić wyjaśnić każdy oddawany fragment, zwłaszcza miejsca decydujące o uruchomieniu programu i interpretacji kodu zakończenia.

📊 Progi i kryteria oceny

Wyższy próg obejmuje wymagania wszystkich niższych progów. Tabela wskazuje maksymalną możliwą ocenę za poprawnie wykonany, udokumentowany i terminowo oddany zakres.

Maksymalna ocena	Wymagany zakres	Minimalne dowody
2	częściowe wykonanie poziomu A	ręczna kompilacja albo jeden zapisany przebieg z rozpoznanym kodem zakończenia
3	poprawnie ukończony poziom A	kompilacja ręczna, dwa automatyczne przebiegi `pass`, katalogi dowodów i wyjaśnienie ich zawartości
4	poziomy A i B	`pass`, `fail`, błąd kompilacji, rozdzielone strumienie, kody zakończenia i dowód, że stary EXE nie został uruchomiony
5	poziomy A–C	dwa działające rozszerzenia, testy ich sukcesu i niepowodzenia oraz pełne dowody
6	poziomy A–C oraz wartościowe rozszerzenie D	wiedza ponadprogramowa, uzasadniona architektura, obsłużone przypadki awaryjne, testy automatu i obrona pracy

Niezależnie od poziomu oceniane są:

- poprawna kolejność kompilacji, kontroli i uruchomienia;
- brak możliwości uruchomienia starego EXE po błędzie budowania;

- właściwa interpretacja `stdout`, `stderr` oraz kodów zakończenia;
- niezmienione, rozdzielone i jednoznacznie przypisane artefakty;
- rozróżnienie awarii środowiska, kompilacji i wykonania;
- zgodność raportu z rzeczywistym kodem i logami;
- anonimizacja, czytelność i oznaczenie dowodów;
- rozumienie oraz umiejętność obrony pracy.

Agent AI wspiera nauczyciela w odnajdywaniu dowodów i porównywaniu raportu z jawnymi kryteriami. Nie wystawia samodzielnie oceny. Materiał przygotowany z pomocą AI, którego nie potraficie zweryfikować albo obronić, nie jest dowodem Waszych umiejętności i może drastycznie obniżyć ocenę, aż do oceny niedostatecznej.

🚧 Typowe pułapki

- uruchomienie programu mimo nieudanego budowania;
- pozostawienie starego EXE i uznanie go za produkt bieżącej kompilacji;
- zapisanie tylko `stdout` i zgubienie diagnostyki ze `stderr`;
- uznanie tekstu `OK` za ważniejszy od kodu zakończenia;
- nadpisywanie jednego pliku logu przy każdym przebiegu;
- ręczne poprawianie logu zamiast opisania problemu w raporcie;
- sklejanie polecenia z niesprawdzonych danych i uruchamianie go przez `shell=True`;
- uruchomienie skryptu w terminalu bez przygotowanego środowiska MSVC;
- uznanie pojedynczego czasu z logu za rzetelny benchmark;
- pomylenie symulowanej awarii z błędem rzeczywiście odkrytym w Bubble Sorcie;
- ujawnienie nazwy konta w ścieżce, zrzucie albo właściwościach pliku;

🏁 Co powinniście zapamiętać?

- Budowanie programu obejmuje co najmniej kompilację i konsolidację.
- Developer PowerShell przygotowuje środowisko potrzebne narzędziom MSVC.
- `stdout`, `stderr` i kod zakończenia są osobnymi źródłami informacji.
- Kod `0` zwykle oznacza sukces, lecz dokładną umowę określa program.
- Po błędzie kompilacji nie wolno uruchamiać starego pliku wykonywalnego.
- Dobry automat zatrzymuje się we właściwym miejscu i zachowuje również dowody niepowodzenia.
- Osobny katalog przebiegu chroni surowe artefakty przed nadpisaniem i pomieszaniem.
- Log jest dowodem, ale nie zastępuje interpretacji w raporcie.
- Skrypt `build → run → evidence` jest lokalnym zalążkiem większej linii CI.
- Automatyzacja zwiększa szybkość i powtarzalność, ale człowiek nadal odpowiada za to, czy procedura sprawdza właściwą rzecz.